CS 486/686
Neural Networks and Learned Embeddings

Yuntian Deng

Lecture 18

From fixed features to learned vector representations

Recorded lecture (asynchronous)

> 🎥 This lecture is **pre-recorded** - I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.
>
> 📅 **Due today (Thu Jul 9):** CS686 project proposal. **Assignment 2** is now due **Thu Jul 16**. Chat 9 is out and Assignment 3 is released.
>
> 💬 Questions? Post on **Piazza** - the TAs are available, and I'll follow up when I'm back.

The problem with fixed features

L17 assumed the useful input vector already exists.

pixels

Where is the cat ear?

word IDs

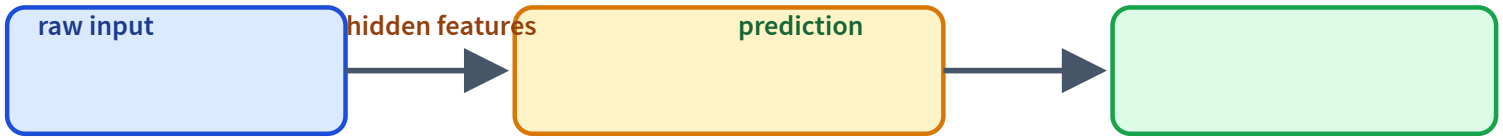
Why is dog close to puppy?

screenshots

Which region is a button?

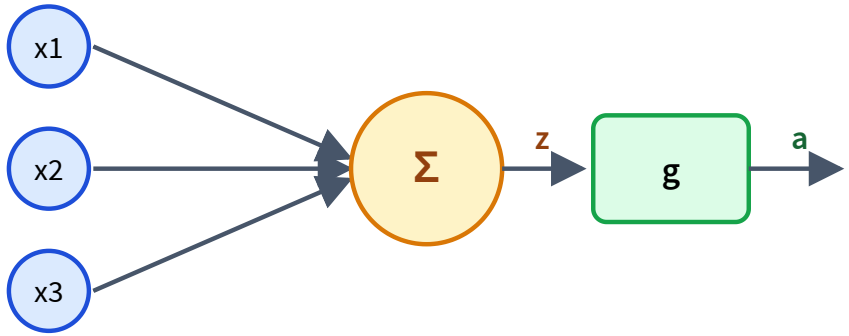
Modern ML learns the features too.

Neural nets learn representations



The hidden layer is not just computation: it is a learned view of the input.

A neuron is a feature detector



weighted sum: $z = w^\top x + b$

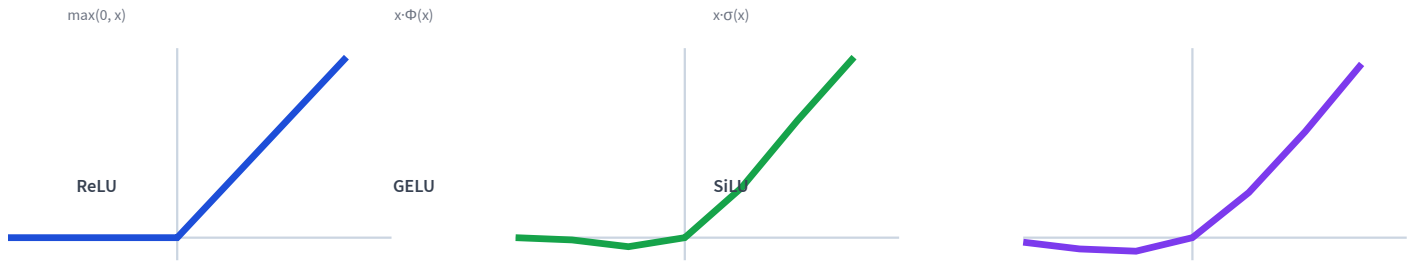
activation: $a = g(z)$

Weights decide what pattern to detect; g adds the nonlinearity.

Why add a nonlinearity?

Stacking linear layers stays linear: $W_2(W_1x) = (W_2W_1)x$ — still one line.

A nonlinearity g between layers is what lets the network bend.



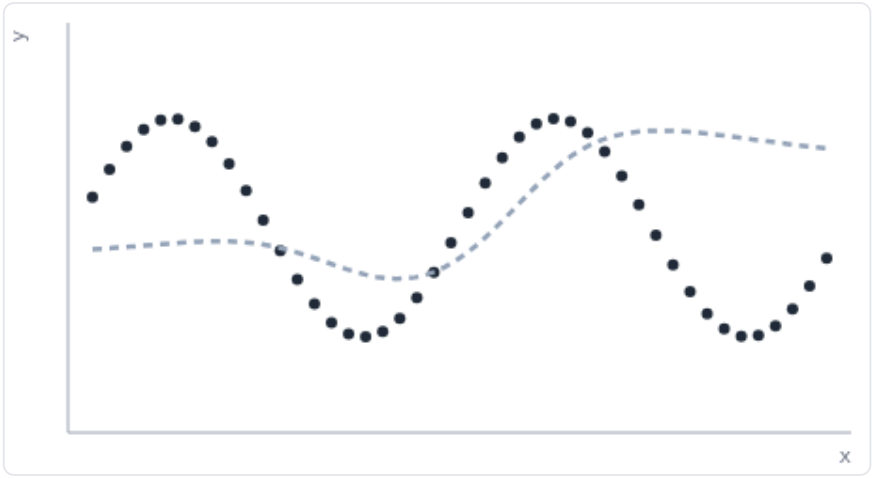
ReLU is the classic default; GELU and SiLU are smooth, near-ReLU curves used in most modern nets and Transformers.

Playground: does nonlinearity matter? LIVE

Interactive demo — runs live in your browser (open the slides to try it).

Model: 1 input → 12 tanh hidden units → 1 output · target y = sin(2x) · trained with the L17 loop

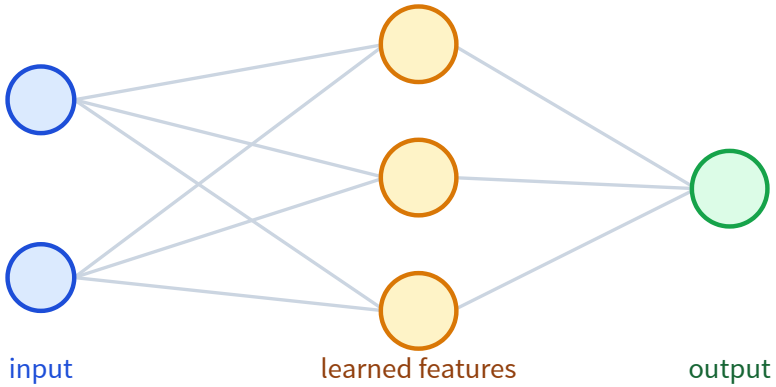
activation: ON Train



activation: **ON (tanh)**
untrained — press **Train**

Same net, same data. Activation OFF: stacked linear layers stay a line. ON: the layer bends to fit the wave. That bend is what nonlinearity buys.

One hidden layer = learned feature map

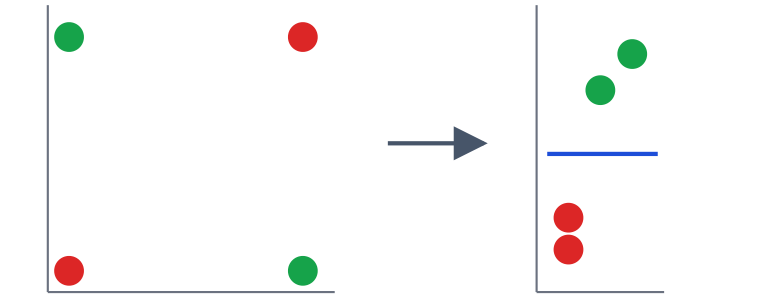


Each hidden unit learns a detector.

The output layer combines detectors.

This is how fixed features become learned features.

XOR: why hidden features help



raw input: not linearly separable

hidden features make it easy

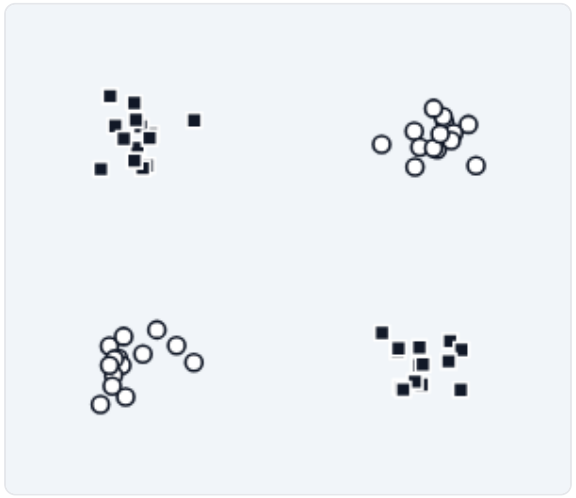
A hidden layer can transform the space.

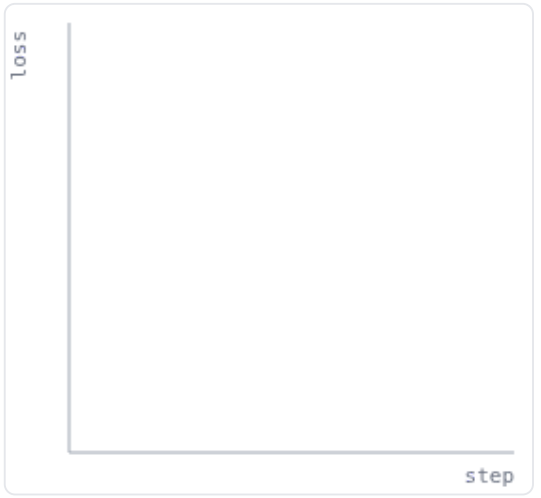
Then a simple output layer can solve the task.

Playground: train a net on XOR LIVE

Interactive demo — runs live in your browser (open the slides to try it).

Train activation: ON





Green/red = the net’s decision surface; markers are the true labels. Turn the activation OFF and the 2-2-1 net becomes linear — it cannot separate XOR no matter how long it trains.

An MLP is a differentiable program

$$h = g(W_1 x + b_1)$$

$$\hat{y} = \mathrm{softmax}(W_2 h + b_2)$$

Every operation is differentiable, so L17's training loop applies unchanged.

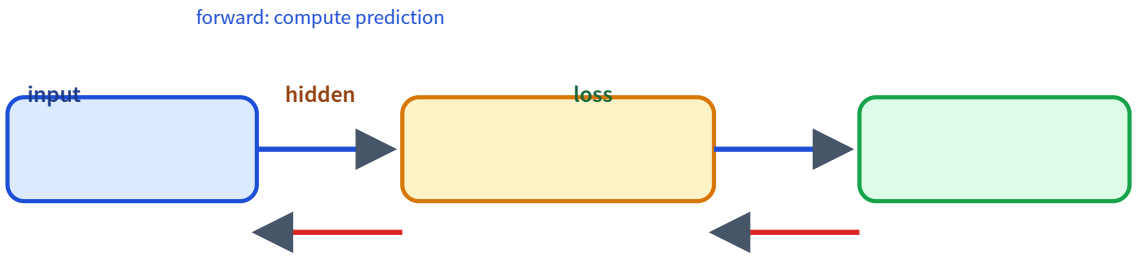
The code barely changes

```
model = nn.Sequential(
    nn.Linear(d_in, 64),
    nn.ReLU(),
    nn.Linear(64, n_classes),
)

loss = loss_fn(model(x), y)
loss.backward()
optimizer.step()
```

The function is richer; the training pattern is the same.

Backpropagation assigns credit



What do hidden layers learn?

Deep models compose simple features into useful abstractions.

CNNs learn visual features

shallow: curves → middle: parts → deep: objects

one deep neuron = a "ball" detector

Feature visualizations (what each neuron responds to most). Olah et al., "Feature Visualization," Distill 2017 (CC-BY).

Local filters detect patterns in small patches.

Weight sharing finds the same pattern anywhere.

Depth composes simple parts into whole objects.

Later: VLMs reuse image encoders to turn images into vectors/tokens.

Word IDs have no geometry

A raw ID is arbitrary:

$$\texttt{cat} = 17, \texttt{dog} = 932$$

An embedding gives each item a vector:

$$\mathrm{Embed}[\mathrm{cat}] \in \mathbb{R}^d$$

Now similarity can live in vector space.

An embedding table is a learnable lookup

It is just a **lookup table** (a dictionary): a token's **id** selects its **row**.

token	id	row = learned vector
cat	17	[0.22, -0.71, 0.10, ...]
dog	932	[0.19, -0.66, 0.12, ...]
car	40	[-0.80, 0.31, -0.40, ...]

`emb[17]` → the "cat" row

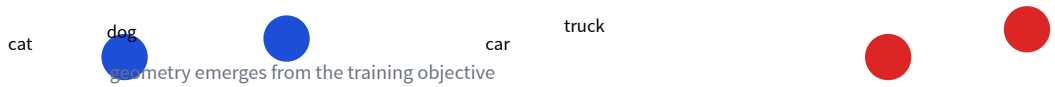
```
emb = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=128,
)
```

Unlike a fixed dictionary, the rows are **parameters**: gradient descent updates them during training.

How does an embedding learn?

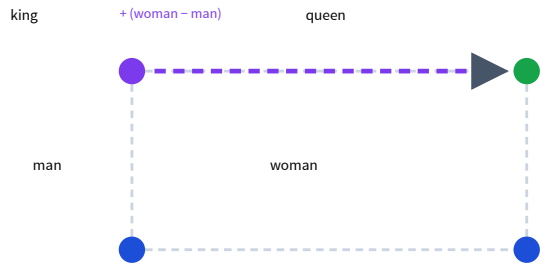
`the cat sat on the ___`

If the model should predict **mat**, gradients update the embeddings that helped make the prediction.



Directions can carry meaning

king − man + woman ≈ queen



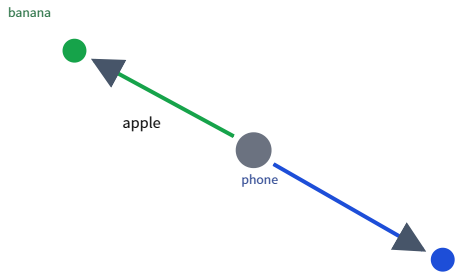
Standalone embeddings — word2vec (Mikolov 2013), GloVe (Pennington 2014) — were trained just to place words, and famously captured such regularities.

Today embeddings are mostly learned **implicitly inside LLMs**; modern contextual vectors rarely do clean arithmetic.

One word can mean two things

"I ate an **apple**" → fruit

"**Apple** released a phone" → company

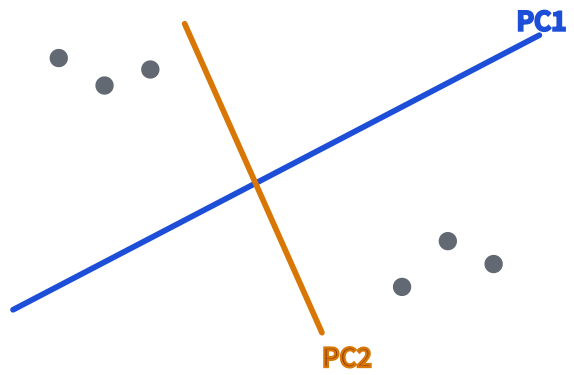


A single fixed vector for "apple" cannot be both.

Modern models embed the **whole sentence**, so a word gets a different **contextual** vector each time.

(This is why the live demo embeds free text, not just single words.)

Inspect embeddings by projecting them



PCA: linear view of high-dimensional vectors.

t-SNE/UMAP: nonlinear visualization.

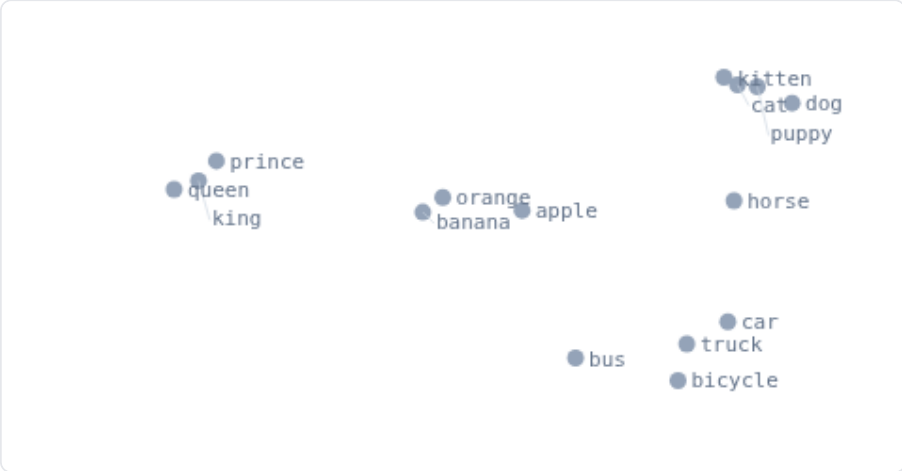
Distances mean what the objective made them mean — a 2D plot can overstate structure, so use it to ask questions, not as proof.

Playground: embed real words LIVE MODEL

Interactive demo — a real embedding model runs in your browser (open the slides to try it).

```
dog, puppy, cat, kitten, horse, car, truck, bus, bicycle, apple, banana,
orange, king, queen, prince
```

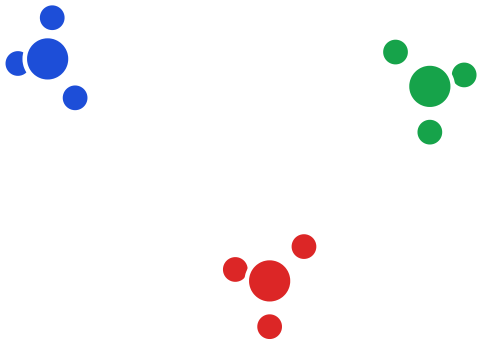
Embed Reset words



click a point to see its nearest neighbors

Type any words or short sentences. MiniLM computes a contextual embedding (not a fixed lookup), then PCA squeezes it to 2D. Similar meanings land near each other.

Cluster embeddings with k-means



After embeddings exist, k-means becomes practical.

- inspect datasets
- find groups or duplicates
- build codebook intuition

The k-means algorithm

1. Choose k; initialize k centroids (e.g. random data points).
2. Repeat until assignments stop changing:
    a. **Assign** — put each point with its nearest centroid.
    b. **Update** — move each centroid to the mean of its points.

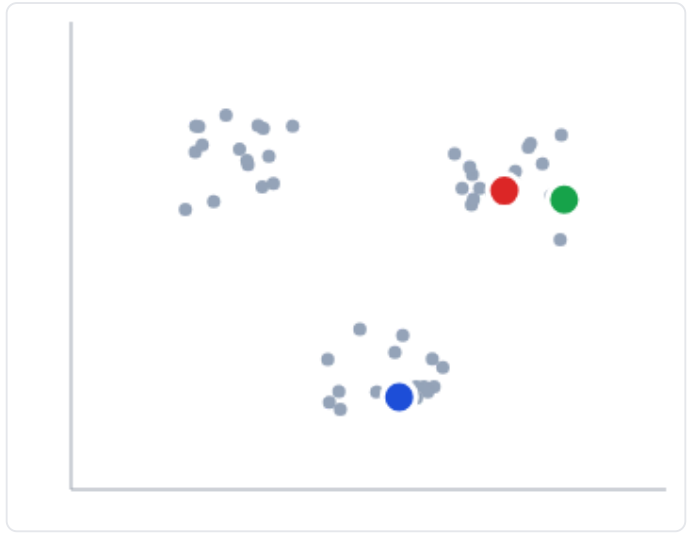
Objective (inertia): $J = \sum_i \left\| x_i - \mu_{c(i)} \right\|^2$ — every step lowers it.

Converges to a **local** optimum, so it depends on initialization — run a few times and keep the best.

Playground: k-means, step by step LIVE

Interactive demo — runs live in your browser (open the slides to try it).

Step Run Reseed



```
iteration 0
centroids placed
```

Each step: assign every point to its nearest centroid, then move each centroid to its members' mean. Repeat until nothing moves.

Embeddings let us measure quality

Embed real and generated samples, then compare the two **distributions** in embedding space.

FID — images

distance between Inception-feature distributions of real vs generated images (lower is better)

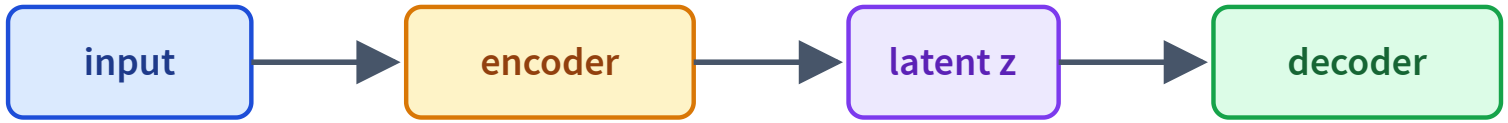
MAUVE — text

compares human vs machine text in a language model's embedding space

Same idea, two modalities: good generations should look like real data *in feature space*.

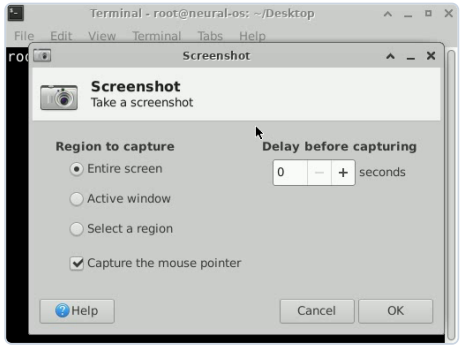
FID: Heusel et al. 2017. MAUVE: Pillutla et al. 2021.

Autoencoders learn compression



Latent diffusion will generate in a compressed latent space.

A real autoencoder: NeuralOS

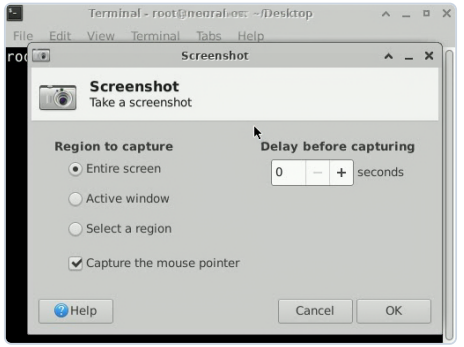


input — 512×384×3

encode →

latent — 64×48×16
(3 of 16 channels)

→ decode



reconstruction

589,824 numbers → 49,152 — a **12×** smaller latent, yet the decode looks the same.

Autoencoder from NeuralOS (neural-os.com).

Why compress? To make generation feasible

neural-os.com

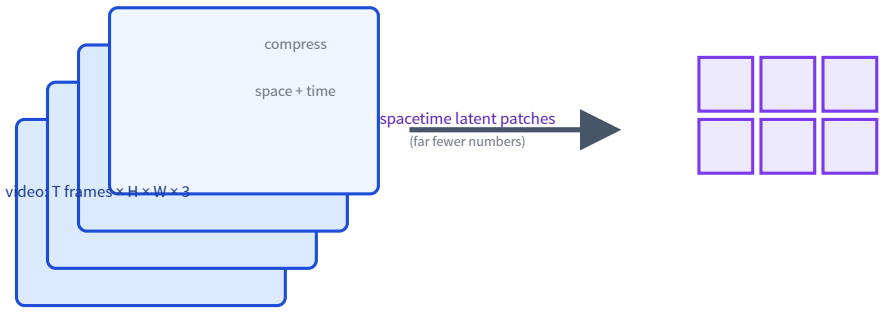
Generating in pixel space is expensive — about 590k numbers *per frame*.

Diffusion runs in the **12× smaller** latent instead, then decodes once at the end.

NeuralOS predicts the next screen frame in latent space — a neural operating system.

Video: compress space and time

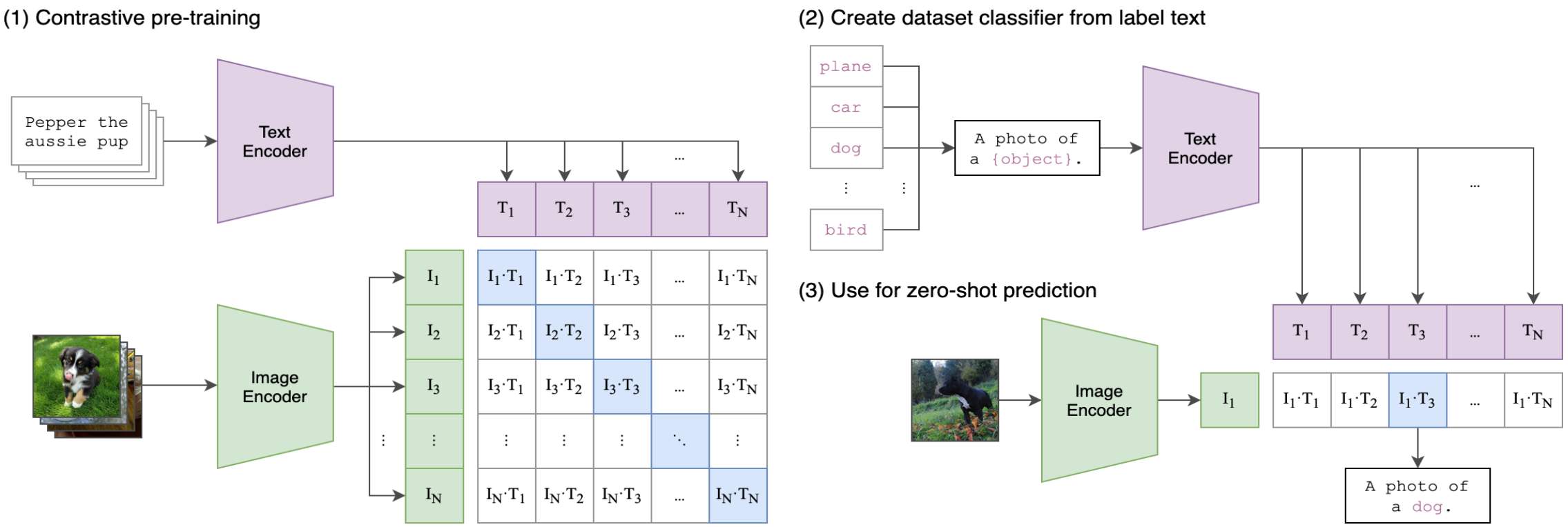
A video is many frames, so pixel cost explodes — compress the time axis too.



Sora compresses space and time into spacetime latent patches, then generates there.

OpenAI, "Video generation models as world simulators," 2024.

Contrastive learning aligns modalities (CLIP)



Train one shared space: matched image/text pairs are pulled together, mismatched pairs pushed apart.

Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML 2021.

What a shared image-text space unlocks

zero-shot classification

score an image against prompts like `"a photo of a {label}"` — no task-specific training

retrieval

search images with text (and vice versa) by nearest vectors in the shared space

This shared space is the bridge to vision-language models (L23).

Why this matters for LLMs

tokens

become embeddings

attention

moves information between embeddings

transformers

scale this up

Recap

✓ Neural nets learn **features**, not just predictions.

✓ Backprop assigns credit through the graph.

✓ **Embeddings** are learned vectors.

✓ PCA, t-SNE, and k-means help inspect embeddings.

✓ Autoencoders and CLIP prepare us for diffusion and VLMs.

Next question

A sentence is a sequence of embeddings.

How should each word look at the other words?

L19: sequence models, attention, and transformers.